

Department of Information Technology
Faculty of Computing
The Islamia University of Bahawalpur



Software Requirement Specifications

Real-Time Face Mask Detection Using Convolutional Neural Networks and OpenCV

Submitted by

Hamid Yasin

F22BINFT1M01095

Submitted to

Mam Sara Fareed

Meeting Details

Sr No	Details	Date	Supervisor Signature

Summary

This project develops a real-time computer vision system capable of automatically detecting whether individuals are wearing face masks. The system uses a labeled dataset containing masked and unmasked faces to train a Convolutional Neural Network (CNN) for binary classification. The pipeline performs essential steps including image preprocessing, data augmentation, and model optimization to ensure high detection accuracy. After training, the model is integrated with an OpenCV-based face detection module that processes live video streams from a webcam. For each detected face, the system predicts mask status and overlays visual feedback using bounding boxes and classification labels. This end-to-end solution provides an efficient and practical tool for real-time monitoring in public spaces, supporting safety compliance and enabling automated surveillance applications.

Table of Contents

1. Introduction	4
1.1. Purpose	4
1.2. Scope	4
1.3. Product Perspective	4
1.4. User Characteristics	5
1.5. Proposed Technologies	5
2. Requirements	5
2.1. Function Requirements	6
2.2. Non-Functional Requirements	7
3. Use Cases and Flow of Processes	8
4. References	9

1. Introduction

This Software Requirements Specification (SRS) document outlines the functional and non-functional requirements, along with the use cases, for the Real-Time Face Mask Detection System. The primary objective of this project is to develop a deep learning–based model capable of determining whether a person is wearing a face mask. The system uses a labeled dataset consisting of masked and unmasked face images, which is preprocessed, augmented, and used to train a Convolutional Neural Network (CNN) for binary classification. After training, the model is integrated with an OpenCV (cv2) pipeline that performs real-time face detection on live video streams, such as webcam. The application identifies faces, classifies each one as “Mask” or “No Mask,” and displays results using bounding boxes and labels. This solution provides an efficient and automated approach to monitoring mask compliance, making it practical for public health, surveillance, and safety applications.

1.1. Purpose

The purpose of this project is to develop a deep learning–based face mask detection system capable of identifying whether individuals are wearing masks in real time. Using a labeled dataset containing masked and unmasked face images, the model will be trained to classify faces into predefined categories. Once the model is trained, it will be integrated with an OpenCV-based video processing pipeline, enabling real-time detection from webcam or CCTV streams. This system aims to support public safety and health monitoring by providing an automated and efficient solution for mask compliance detection.

1.2. Scope

The scope of this project involves developing a real-time face mask detection system using a labeled dataset of masked and unmasked face images. While the CNN model may leverage pre-trained architectures through transfer learning to improve accuracy and reduce training time, it will be fine-tuned for the specific task of mask detection. The trained model will be integrated with an OpenCV-based video processing pipeline, enabling real-time detection from webcams. The system will automatically identify faces, classify them as “Mask” or “No Mask,” and display the results with bounding boxes and labels, providing a practical solution for automated mask compliance monitoring in public and private spaces.

1.3. Product Perspective

The developed plant disease classification module will be an integral component of a larger agricultural web application. It will serve as a machine learning-driven backend service, where users can upload plant leaf images to receive real-time disease classifications. The module will interact seamlessly with the front-end Streamlit web interface, providing an easy-to-use platform for farmers or users to diagnose plant health. The model, once trained and fine-tuned, will be accessible via the web app, thereby facilitating efficient and automated disease detection.

1.4. User Characteristics

End-users, such as security personnel, facility managers, or the general public, will interact with the system through a real-time video interface to monitor mask compliance. The system is designed to be intuitive and user-friendly, ensuring that users with minimal technical knowledge can easily operate it. With a few simple steps, users can start a webcam or connect to a CCTV feed, and the system will automatically detect faces and classify them as “Mask” or “No Mask” in real time. The goal is to provide an accessible tool that supports public health and safety monitoring without requiring specialized expertise in computer vision or machine learning.

1.5. Proposed Technologies

The proposed project will leverage the following technologies:

1. **Deep Learning (YOLOv8):** A YOLOv8n-based convolutional neural network for accurate and efficient face mask detection in real-time video streams.
2. **OpenCV (cv2):** For capturing and processing live video frames from webcams or CCTV cameras, and for integrating the face detection and annotation pipeline.
3. **Python:** The primary programming language for developing the system, using libraries such as Tensorflow for model handling, NumPy for data processing, and OpenCV for video stream management.
4. **Kaggle Dataset:** A dataset containing images of masked and unmasked faces, used to train and validate the deep learning model for binary classification.

2. Requirements

The system will allow users to monitor live video streams from webcams or CCTV cameras, after which the deep learning model will detect faces in real time and classify each as either “Mask” or “No Mask.” Users can view the results directly on the video feed, with bounding boxes and labels indicating the classification for each detected face. The system will handle frame capturing and preprocessing to ensure accurate and consistent detection across varying lighting conditions and video resolutions. Additionally, the application will provide a real-time, interactive, and user-friendly interface, enabling users to continuously monitor multiple faces without requiring technical expertise. The model will be trained on a labeled dataset of masked and unmasked faces to achieve high accuracy and generalization across different individuals, face angles, and environments.

2.1. Function Requirements

The functional requirements define the essential features and operations that the system must perform to achieve its objectives. These requirements focus on the key functions of the real-time face mask detection system. The system will enable users to monitor live video streams, detect faces, and classify each detected face as wearing a mask or not. The classification results will be displayed in real time with bounding boxes and labels, providing a clear and interactive interface. The system will handle frame capturing and preprocessing to ensure accurate detection. Additionally, the model will be trained on a labeled dataset of masked and unmasked faces to ensure high accuracy and generalization. Overall, these functional requirements form the backbone of the project, guiding development and ensuring all critical tasks are efficiently performed.

2.1.1. Face Detection and Classification

- **Id:** FR001
- **Title:** Face Detection and Classification
- **Description:** *The system shall detect faces in live video streams and classify each face as “Mask” or “No Mask” using the trained deep learning model.*

2.1.2. Display Detection Results

- **Id:** FR002
- **Title:** Display Detection Results

- **Description:** *The system shall overlay bounding boxes and labels on detected faces in the video stream, showing the classification result in real time.*

2.1.3. Frame Preprocessing

- **Id:** *FR003*
- **Title:** *Frame Preprocessing*
- **Description:** *The system shall preprocess video frames, including resizing, normalization, and face alignment, to ensure accurate detection and classification.*

2.1.4. User Interaction

- **Id:** *FR004*
- **Title:** *User Interaction*
- **Description:** *The system shall provide a user-friendly interface allowing users to upload multiple images and view predictions in real-time.*

2.1.5. Model Training

- **Id:** *FR005*
- **Title:** *Model Training*
- **Description:** *The system shall allow training of the YOLOv8n model on a labeled dataset of masked and unmasked faces to ensure robust performance and accurate classification.*

2.2. Non-Functional Requirements

*Non-functional requirements define the overall qualities and constraints of the system, focusing on how the face mask detection system performs rather than what it performs. These requirements ensure that the system is reliable, efficient, and user-friendly. The system should be highly **responsive**, providing real-time detection and classification results with minimal delay when processing video streams. It must be **scalable**, capable of handling multiple faces and video streams simultaneously without performance degradation. **Accuracy and reliability** of the predictions are critical, ensuring that the YOLOv8n model produces consistent and trustworthy results across different individuals, face angles, and environments. The system should maintain a high level of **usability**, offering an intuitive interface that requires minimal technical knowledge to operate. Additionally, the application must adhere to **maintainability and extensibility**,*

allowing future updates to the model, video pipeline, or interface with ease. **Security and privacy** of any video feeds or user data must also be ensured, even if no sensitive personal information is stored. Overall, these non-functional requirements guide the design and deployment of the system to provide a robust, efficient, and user-friendly real-time mask detection solution.

3. Use Cases and Flow of Processes

Use cases describe how users interact with a system to achieve specific goals, translating functional requirements into practical scenarios. In this project, the primary actors are end-users who monitor live video streams and the system, which handles frame capturing, face detection, mask classification, and result display. These use cases help the development team understand critical processes and allow stakeholders to visualize the system's operation in real-world scenarios.

3.1. Real-Time Face Detection and Classification

ID	UC001
Name	Real-Time Face Detection and Classification
Requirement(s)	FR001,FR003,FR005
Actor(s)	User
Precondition	<i>The user has access to the system and a live video feed from a webcam or CCTV camera is available.</i>
Postcondition	<i>Faces are detected, and each face is classified as "Mask" or "No Mask."</i>
Basic Flow	<i>1. The user starts the video stream.</i> <i>2. The system captures frames from the video feed.</i> <i>3. The system detects faces in each frame using YOLOv8n.</i> <i>4. Each detected face is classified as "Mask" or "No Mask."</i> <i>5. Classification results are prepared for display.</i>

3.2. Display Detection Results

ID	UC002
Name	Display Detection Results
Requirement(s)	FR002,FR003, FR004,FR005
Actor(s)	User, System
Precondition	<i>Faces have been detected and classified.</i>
Postcondition	<i>Users can view real-time detection results with bounding boxes and labels.</i>
Basic Flow	1. The system overlays bounding boxes around each detected face. 2. The system adds labels indicating “Mask” or “No Mask” for each face. 3. Users view the results in real time on the video feed. 4. Users can continue monitoring without interruption.

3.3. Repeat Monitoring

ID	UC003
Name	Repeat Monitoring
Requirement(s)	FR001,FR002,FR003, FR004, FR005
Actor(s)	User
Precondition	<i>The system has completed at least one frame analysis.</i>
Postcondition	<i>Users can continuously monitor new frames and video feeds without restarting the application.</i>
Basic Flow	1. The system continues capturing frames from the live video stream. 2. Each new frame is processed for face detection and mask classification. 3. Updated results are displayed in real time. 4. Users can monitor multiple individuals continuously.

4. References

1. O. Gurav, "**Face Mask Dataset**," Kaggle, 2020.
<https://www.kaggle.com/datasets/omkargurav/face-mask-dataset>.
2. H. Yasin, "**Real-Time Face Mask Detection Using Convolutional Neural Networks and OpenCV**," GitHub Repository.
<https://github.com/hamidyaseen644/Face-mask-detection->
3. TensorFlow, "**Keras Guide**," TensorFlow Documentation.
<https://www.tensorflow.org/guide/keras>.
4. Streamlit, "**Streamlit Documentation**," Streamlit Web Tool. <https://docs.streamlit.io/>.
5. Google, "**Colaboratory**," Google Colab. <https://colab.google/>.